

EXPERIMENT - 4

AIM

Using the Employee schema, write and execute SQL queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare the execution plans of selected queries using EXPLAIN.

OBJECTIVES

- To understand how different types of JOIN operations combine rows from related tables.
- To perform a self-join on the Employee table to compare employees within the same department.
- To use correlated subqueries and EXISTS for row-by-row and existence-based filtering.
- To simulate INTERSECT and EXCEPT operations using subqueries supported by MySQL.
- To inspect and compare query execution plans using EXPLAIN.

DATABASE USED

USE CompanyDB;

SCHEMA REFERENCE

Table	Important Columns	Relationship
Employee	emp_id, emp_name, salary, hire_date, dept_id, project_id	dept_id → Department; project_id → Project
Department	dept_id, dept_name	Primary key: dept_id
Project	project_id, project_name, budget, dept_id	Primary key: project_id; dept_id → Department

SOURCE CODE / SQL QUERIES

1. INNER JOIN

Display employee names, salaries and their department names using an INNER JOIN.

```
SELECT
    e.emp_id,
    e.emp_name,
    e.salary,
    d.dept_name
FROM Employee e
INNER JOIN Department d
    ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

Output

I/O AI Agent 108 ms

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	salary	dept_name
1	Ishaan	65000.00	Development
2	Reyansh	72000.00	Development
3	Adit	58000.00	Development
4	Aarush	81000.00	Development
5	Raghav	69000.00	Development
6	Vihaan	62000.00	Development
7	Anika	52000.00	PeopleOps
8	Kavya	48000.00	PeopleOps
9	Ira	61000.00	PeopleOps
10	Naina	55000.00	PeopleOps
11	Aditi	47000.00	PeopleOps
12	Saanvi	59000.00	PeopleOps
13	Harsh	75000.00	Accounts
14	Mohit	68000.00	Accounts
15	Ritvik	83000.00	Accounts
16	Ayush	62000.00	Accounts
17	Kunal	71000.00	Accounts
18	Varun	56000.00	Accounts
19	Mehak	54000.00	Sales
20	Pihu	63000.00	Sales
21	Shreya	57000.00	Sales
22	Tara	66000.00	Sales
23	Mahi	51000.00	Sales
24	Anvi	70000.00	Sales
25	Rajat	60000.00	Logistics
26	Yuvraj	73000.00	Logistics
27	Arnav	67000.00	Logistics
28	Laksh	59000.00	Logistics
29	Devansh	76000.00	Logistics
30	Om	64000.00	Logistics

2. LEFT JOIN

```
SELECT
    d.dept_id,
    d.dept_name,
    e.emp_name,
    e.salary FROM Department d
LEFT JOIN Employee e
    ON d.dept_id = e.dept_id
ORDER BY d.dept_id, e.emp_id;
```

OUTPUT

I/O AI Agent 116 ms

STDIN

Input for the program (Optional)

Output:

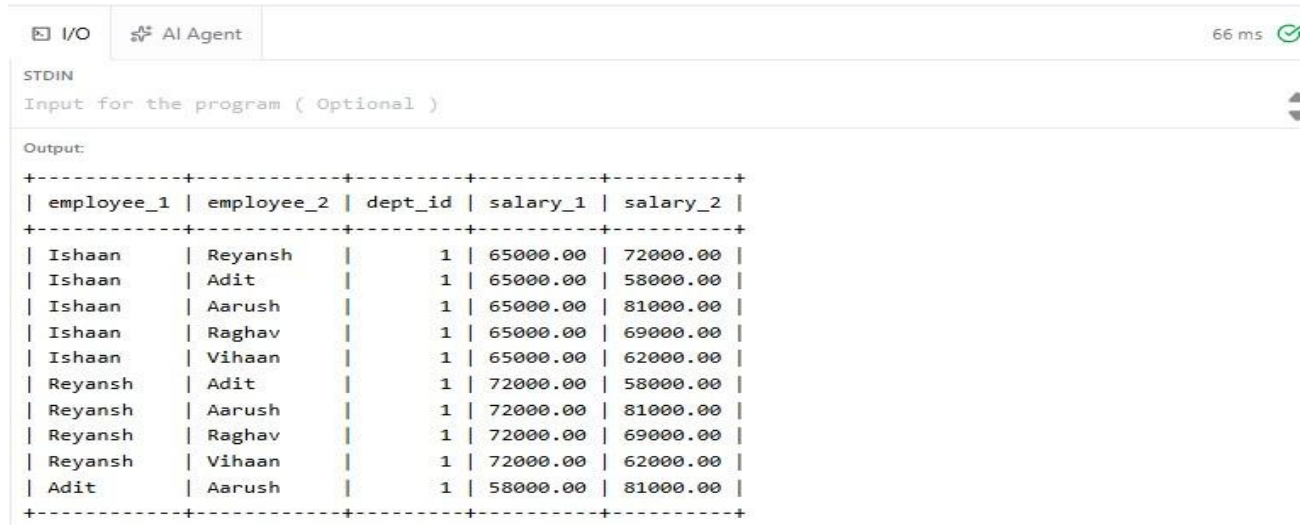
dept_id	dept_name	emp_name	salary
1	Development	Ishaan	65000.00
1	Development	Reyansh	72000.00
1	Development	Adit	58000.00
1	Development	Aarush	81000.00
1	Development	Raghav	69000.00
1	Development	Vihaan	62000.00
2	PeopleOps	Anika	52000.00
2	PeopleOps	Kavya	48000.00
2	PeopleOps	Ira	61000.00
2	PeopleOps	Naina	55000.00
2	PeopleOps	Aditi	47000.00
2	PeopleOps	Saanvi	59000.00
3	Accounts	Harsh	75000.00
3	Accounts	Mohit	68000.00
3	Accounts	Ritvik	83000.00
3	Accounts	Ayush	62000.00
3	Accounts	Kunal	71000.00
3	Accounts	Varun	56000.00
4	Sales	Mehak	54000.00
4	Sales	Pihu	63000.00
4	Sales	Shreya	57000.00
4	Sales	Tara	66000.00
4	Sales	Mahi	51000.00
4	Sales	Anvi	70000.00
5	Logistics	Rajat	60000.00
5	Logistics	Yuvraj	73000.00
5	Logistics	Arnav	67000.00
5	Logistics	Laksh	59000.00
5	Logistics	Devansh	76000.00
5	Logistics	Om	64000.00

3. SELF-JOIN

Compare two employees belonging to the same department. The Employee table is joined to itself using two aliases.

```
SELECT
    e1.emp_name AS employee_1,
    e2.emp_name AS employee_2,
    e1.dept_id,
    e1.salary AS salary_1,
    e2.salary AS salary_2
FROM Employee e1
INNER JOIN Employee e2
    ON e1.dept_id = e2.dept_id
    AND e1.emp_id < e2.emp_id
ORDER BY
    e1.dept_id,
    e1.emp_id,
    e2.emp_id
LIMIT 10;
```

OUTPUT (first 10 rows)



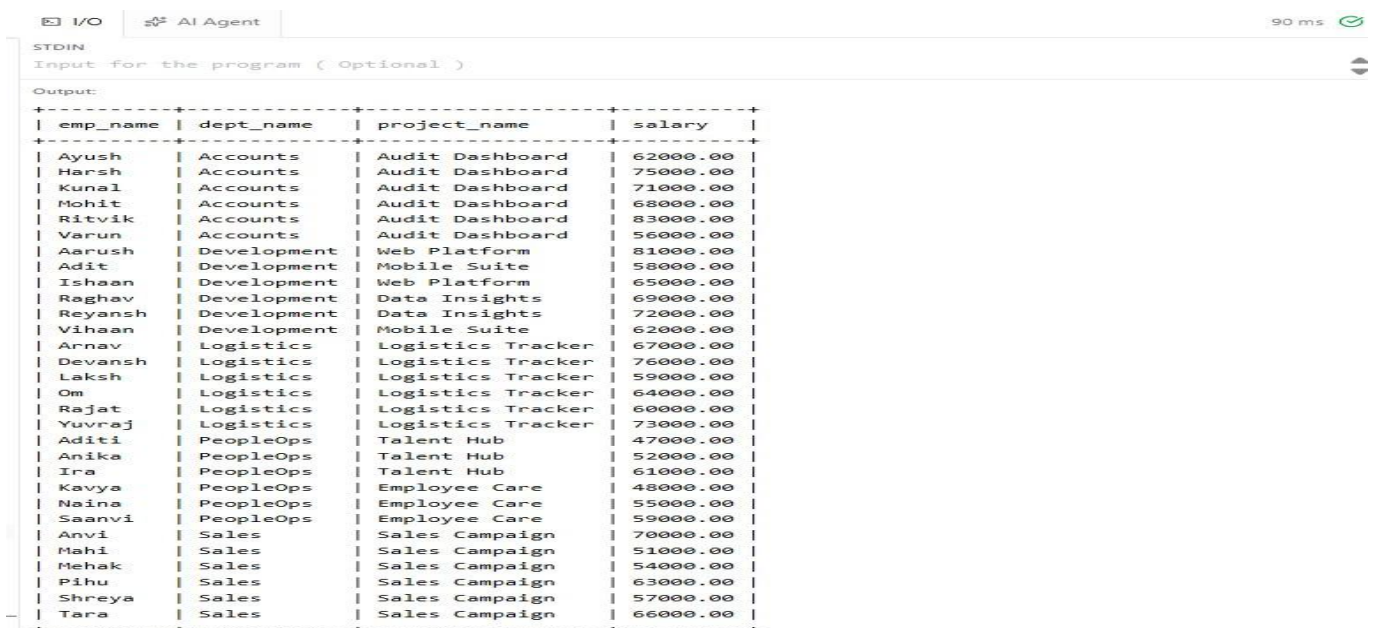
employee_1	employee_2	dept_id	salary_1	salary_2
Ishaan	Reyansh	1	65000.00	72000.00
Ishaan	Adit	1	65000.00	58000.00
Ishaan	Aarush	1	65000.00	81000.00
Ishaan	Raghav	1	65000.00	69000.00
Ishaan	Vihaan	1	65000.00	62000.00
Reyansh	Adit	1	72000.00	58000.00
Reyansh	Aarush	1	72000.00	81000.00
Reyansh	Raghav	1	72000.00	69000.00
Reyansh	Vihaan	1	72000.00	62000.00
Adit	Aarush	1	58000.00	81000.00

4. 3-WAY JOIN

Display employee name, department, project and salary by joining Employee, Department and Project.

```
SELECT
    e.emp_name,
    d.dept_name,
    p.project_name,
    e.salary
FROM Employee e
INNER JOIN Department d
    ON e.dept_id = d.dept_id
INNER JOIN Project p
    ON e.project_id = p.project_id
ORDER BY d.dept_name, e.emp_name;
```

OUTPUT



emp_name	dept_name	project_name	salary
Ayush	Accounts	Audit Dashboard	62000.00
Harsh	Accounts	Audit Dashboard	75000.00
Kunal	Accounts	Audit Dashboard	71000.00
Mohit	Accounts	Audit Dashboard	68000.00
Ritvik	Accounts	Audit Dashboard	83000.00
Varun	Accounts	Audit Dashboard	56000.00
Aarush	Development	Web Platform	81000.00
Adit	Development	Mobile Suite	58000.00
Ishaan	Development	Web Platform	65000.00
Raghav	Development	Data Insights	69000.00
Reyansh	Development	Data Insights	72000.00
Vihaan	Development	Mobile Suite	62000.00
Annav	Logistics	Logistics Tracker	67000.00
Devansh	Logistics	Logistics Tracker	76000.00
Laksh	Logistics	Logistics Tracker	59000.00
Om	Logistics	Logistics Tracker	64000.00
Rajat	Logistics	Logistics Tracker	60000.00
Yuvraj	Logistics	Logistics Tracker	73000.00
Aditi	PeopleOps	Talent Hub	47000.00
Anika	PeopleOps	Talent Hub	52000.00
Ira	PeopleOps	Talent Hub	61000.00
Kavya	PeopleOps	Employee Care	48000.00
Naina	PeopleOps	Employee Care	55000.00
Saanvi	PeopleOps	Employee Care	59000.00
Anvi	Sales	Sales Campaign	70000.00
Mahi	Sales	Sales Campaign	51000.00
Mehak	Sales	Sales Campaign	54000.00
Pihu	Sales	Sales Campaign	63000.00
Shreya	Sales	Sales Campaign	57000.00
Tara	Sales	Sales Campaign	66000.00

5. CORRELATED SUBQUERY

Display employees whose salary is greater than the average salary of their own department. The inner query refers to the outer Employee row, making it a correlated subquery.

```
SELECT
    e.emp_id,
    e.emp_name,
    e.salary,
    e.dept_id FROM Employee e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id = e.dept_id
)
ORDER BY e.dept_id, e.salary DESC;
```

OUTPUT

I/O AI Agent 121 ms

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	salary	dept_id
4	Aarush	81000.00	1
2	Reyansh	72000.00	1
5	Raghav	69000.00	1
9	Ira	61000.00	2
12	Saanvi	59000.00	2
10	Naina	55000.00	2
15	Ritvik	83000.00	3
13	Harsh	75000.00	3
17	Kunal	71000.00	3
24	Anvi	70000.00	4
22	Tara	66000.00	4
20	Pihu	63000.00	4
29	Devansh	76000.00	5
26	Yuvraj	73000.00	5
27	Arnav	67000.00	5

6. EXISTS

Find departments that have at least one employee with a salary greater than 80,000.

```
SELECT
    d.dept_id,
FROM Department d
WHERE EXISTS (
    SELECT 1
    FROM Employee e
    WHERE e.dept_id = d.dept_id
    AND e.salary > 80000
)
ORDER BY d.dept_id;
```

OUTPUT

I/O

AI Agent

91 ms

STDIN

Input for the program (Optional)

Output:

```
+-----+
| dept_id | dept_name |
+-----+
|      1 | Development |
|      3 | Accounts   |
+-----+
```

7. SIMULATED INTERSECT

```
SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id IN (
    SELECT emp_id
    FROM Employee
    WHERE dept_id = 1
)
AND emp_id IN (
    SELECT emp_id
    FROM Employee
    WHERE salary > 70000
)
ORDER BY emp_id;
```

Output

I/O

AI Agent

109 ms

STDIN

Input for the program (Optional)

Output:

emp_id	emp_name	salary
2	Reyansh	72000.00
4	Aarush	81000.00

8. SIMULATED EXCEPT

```
SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id IN (
    SELECT emp_id
    FROM Employee
    WHERE salary > 70000
)
AND emp_id NOT IN (
    SELECT emp_id
```

```

FROM Employee
WHERE dept_id = 1
)
ORDER BY emp_id;

```

I/O		AI Agent	102 ms	✓
STDIN				
Input for the program (Optional)				
Output:				
+-----+-----+-----+				
emp_id emp_name salary				
+-----+-----+-----+				
13 Harsh 75000.00				
15 Ritvik 83000.00				
17 Kunal 71000.00				
26 Yuvraj 73000.00				
29 Devansh 76000.00				
+-----+-----+-----+				

9. EXPLAIN – EXECUTION PLAN

EXPLAIN shows how the MySQL optimizer intends to execute a SELECT statement. It can be used to inspect access type, possible indexes, selected index, estimated rows and additional operations.

A. EXPLAIN for INNER JOIN

```

EXPLAIN
SELECT
    e.emp_name,
    d.dept_name FROM Employee e
INNER JOIN Department d
    ON e.dept_id = d.dept_id
WHERE e.salary > 70000;

```

B. EXPLAIN for Correlated Subquery

```

EXPLAIN
SELECT
    e.emp_name,
    e.salary FROM Employee e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.dept_id = e.dept_id
);

```

C. EXPLAIN for EXISTS

```

EXPLAIN
SELECT d.dept_name
FROM Department d
WHERE EXISTS (
    SELECT 1
    FROM Employee e
    WHERE e.dept_id = d.dept_id
    AND e.salary > 80000
);

```

10. COMPARISON OF EXECUTION PLANS

Query	Main Operation	Plan Aspect to Compare	Expected Observation
INNER JOIN	Employee ↔ Department	Join access on dept_id	Checks how Employee and Department are accessed and joined.
Correlated Subquery	Outer Employee + repeated department AVG logic	Subquery execution and rows examined	May require more work because the inner query is logically related to each outer row.
EXISTS	Department + existence test on Employee	Early existence check and filtering	Can stop searching once a qualifying Employee row is found.

RESULT

The Employee–Department–Project database was used to successfully demonstrate INNER JOIN, LEFT JOIN, self-join, 3-way JOIN, correlated subqueries, EXISTS, simulated INTERSECT and simulated EXCEPT. EXPLAIN statements were also prepared to compare how MySQL plans the different query forms.

CONCLUSION

Different SQL constructs can produce related results while using different execution strategies. JOIN operations combine rows from tables, correlated subqueries evaluate related values for outer rows, EXISTS tests whether matching rows exist, and subqueries can simulate set operations such as INTERSECT and EXCEPT. EXPLAIN provides information needed to study and compare these execution strategies.